BCSE410L – Cyber Security Project

PROMPT INJECTION DEFENSE FOR LLM AGENTS

A Middleware Layer for Detecting and Neutralising Prompt

Injection Attacks in Tool-Using LLM Agents

23BCE0241

SANCHIT BAGHEL

23BCE0794

DEV SINGH

Under the Supervision of

Dr Anto S

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering

October 2026

ABSTRACT

A large language model reads everything as one flat stream of tokens. Nothing in the architecture marks which part the developer wrote and which part came out of a web page the developer asked it to fetch. Once that model is given tools such as file access, browsing, mail and a shell, it will act on whatever text it happens to read. Anyone who can get words in front of the agent can therefore issue commands under the victim's own identity. The OWASP Top 10 for Large Language Model Applications ranks this class of attack, indirect prompt injection, at LLM01.

This project presents PID-Guard, a model-agnostic middleware layer that separates instruction from data outside the model, in a place the attacker's text cannot reach. Untrusted text passes through five stages. Canonicalisation reverses zero-width characters, homoglyph substitution, hidden markup and layered encodings. Five detectors then score the text over overlapping windows: signature rules, a cross-lingual lexicon, a logistic-regression classifier, nearest-neighbour similarity and structural analysis. Their evidence is fused by noisy-OR and weighted by the channel the text arrived on. Suspicious spans are datamarked and fenced instead of deleted, so the agent keeps the information it needs. An egress guard then checks the outbound path and enforces a capability gate that lasts one turn.

On a held-out set of 225 samples the system attains 94.3% attack recall at a 2.9% false-positive rate (F1 0.948), adding 5.644 milliseconds per span with no third-party dependency. A cross-lingual benchmark of 108 samples across 12 languages raised recall from 2.1% to 100.0% with no false positives on foreign-language controls, closing a complete bypass. Against an adaptive white-box adversary the system retains 86.7% robustness.

Keywords – Prompt Injection, Large Language Models, LLM Agents, Indirect Injection, AI Security, Adversarial Robustness, Defence in Depth, Natural Language Processing.

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	i
1.	INTRODUCTION	1
	1.1 Background	1
	1.2 Motivation	1
	1.3 Scope of the Project	1
2.	PROJECT DESCRIPTION AND GOALS	3
	2.1 Literature Review	3
	2.2 Research Gap	4
	2.3 Problem Statement	4
	2.4 Objectives	5
	2.5 Project Plan	5
3.	TECHNICAL SPECIFICATION	7
	3.1 Requirements	7
	3.2 Feasibility Study	8
	3.3 System Specification	9
4.	DESIGN APPROACH AND DETAILS	11
	4.1 System Architecture	11
	4.2 Design	12
5.	EXPERIMENTAL SETUP AND OUTPUT	16
	5.1 Experimental Setup	16
	5.2 Output of the Proposed System	16
6.	RESULT ANALYSIS	19
	6.1 Comparison with Existing Systems	19
7.	CONCLUSION AND FUTURE WORKS	22
8.	REFERENCES	23
	APPENDIX A. SOURCE CODE AND REPOSITORY	25
	APPENDIX B. CORE IMPLEMENTATION	27

1. INTRODUCTION

1.1 Background

Large language models are no longer used only to generate text. They are increasingly deployed as agents that retrieve documents, browse the web, read mail and call tools on a user's behalf. That changes the security picture. A passive model that writes something undesirable does limited harm. An agent that can read files, send mail and make network requests does real harm when misled.

The problem is structural, not a bug in any one product. A model receives its system prompt, the user's request and any retrieved content as one undifferentiated token sequence, with no field or delimiter marking which portion carries authority. To the model, text pulled out of a document looks exactly like an instruction from its operator.

Indirect prompt injection exploits exactly this. The attacker never contacts the agent at all. They put crafted text where the agent will later read it: a support ticket, an HTML comment no human will see, a CV in an automated screening pipeline. The planted instruction then competes with the legitimate task, and it often wins. OWASP ranks prompt injection as LLM01, the top risk for these applications, and notes that no complete fix exists at the model layer.

1.2 Motivation

Most proposed defences share one weakness. They live inside the prompt, which is the same channel the attack uses. Telling a model to ignore commands it finds in documents puts a defensive sentence up against an offensive one and leaves the model to referee. It also fails silently. When the defence is overridden, the operator gets no signal at all, arguably worse than no defence.

Two other approaches are no better. Keyword blocklists are easy to get around by rewording: we measured one, and it caught only 2.3% of attacks written without the stock phrasing. Using a second language model as a judge doubles the cost and delay of every request, and the judge is itself a model reading attacker-controlled text, so it can be injected too.

We started from a simple position: the control has to sit outside the model. If the model cannot reliably tell an instruction from data, something else should make that call for it. That something should be deterministic, should track where every span came from, and should be out of reach of the attacker's text. Such a layer can be unit-tested and audited, which a paragraph of English in a system prompt cannot.

1.3 Scope of the Project

The project delivers a working middleware library that sits between an agent runtime and a language model and controls traffic in both directions. On the way in it canonicalises, inspects, scores and neutralises untrusted text. On the way out it checks the model's output

and any tool call it proposes for leaked credentials, exfiltration channels and capability violations.

In scope: the eight attack families named in the literature, resistance to character-level and encoding-level evasion, cross-lingual coverage across twelve languages, a trust model with five levels, per-turn capability restriction, an append-only audit trail, a reproducible evaluation that includes an adaptive adversary, and an interactive console for demonstrating the system against a local model.

Out of scope: changing or fine-tuning the model itself, defending against an attacker who can reach the model weights or our own configuration file, and attacks that never travel through the text channel, such as a compromised tool implementation. We designed this as one layer of a defence-in-depth setup, not as a complete answer, and we report it on that basis.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Perez and Ribeiro (2022) gave the first systematic account of the problem. They named the two primitives that are still the standard taxonomy: goal hijacking, where the attacker pulls the model off the operator's task, and prompt leaking, where the attacker extracts the confidential system prompt. They also showed that handcrafted adversarial prompts work across different model families, so the weakness is not an artefact of one training run.

Greshake et al. (2023) introduced the indirect variant that this project is about. Their key point is that the attacker never has to interact with the agent at all. It is enough to poison a document the agent will later retrieve. They catalogue delivery routes including web pages, mail and code repositories, and they demonstrate working compromises of deployed retrieval-augmented systems. What they contribute is a threat model, not a defence.

Liu et al. (2024) gave the field its first shared formalisation and benchmark, so attacks and defences could finally be compared on equal terms. Their main empirical finding matters directly to us. Prompt-level defences fall off sharply when the attack is paraphrased, because defence and attack sit in the same channel and the model has to choose between them.

Hines et al. (2024) proposed spotlighting, a set of techniques that mark untrusted content so the model can tell it apart from instructions. The most effective variant was datamarking, which threads a private marker character through the untrusted span. We implement datamarking directly. It has one weakness we want to be explicit about: it depends on the model choosing to respect the convention. A model that ignores the marker is simply unprotected, and nothing tells the operator that this has happened.

Willison (2023) described the dual LLM pattern. A quarantined model handles the untrusted content and holds no privileges, while a privileged model never sees that content directly. The isolation is architecturally sound, but it is expensive. It needs two inference passes and a tightly constrained interface between them, which puts it out of reach for most deployments.

Zou et al. (2023) showed that adversarial suffixes found by gradient search transfer across model families. Two things follow. Susceptibility is a property of the architecture, not of one training run, and any defence built on a fixed blocklist will not hold. Carlini et al. (2023) reached a matching conclusion from the alignment side, showing that models aligned against harmful output are still exploitable by an adversary. Between them, these two results are why we put our control outside the model instead of trying to harden the model.

Yi et al. (2023) built a benchmark specifically for indirect injection and tested several defences on it. They found that detectors trained on one attack distribution generalise poorly to another. That finding is what led us to calibrate by leave-one-family-out, withholding a whole attack category and then measuring recall on it. Debenedetti et al. (2024) introduced AgentDojo, where attacks and defences are evaluated against an agent doing realistic tasks

instead of against static strings. Their insistence on measuring usefulness alongside security is why we report the false-positive rate and task completion next to every detection figure.

Abdelnabi et al. (2023) catalogued delivery vectors and showed that retrieval pipelines, mail and code repositories all work as carriers. That survey is where our five trust tiers come from. On the evasion side, Li et al. (2019) showed with TextBugger that small character-level changes defeat text classifiers while a human still reads the text normally. That result drives two of our design choices: the canonicalisation stage, and the use of character n-grams alongside word n-grams in the learned detector. The Unicode Consortium (2023) security-mechanisms specification supplies the confusable mappings behind our homoglyph table.

The OWASP Foundation (2023) supplies the risk framing and the LLM01 classification we use throughout this report. The wider literature on canary tokens and exfiltration through rendered markup informs our egress stage, and in particular the zero-click image beacon, where a markdown image reference makes the client issue a network request with no user interaction at all.

2.2 Research Gap

The surveyed work falls into three groups, and none of them gives you something you can actually deploy. The first group (Perez and Ribeiro, Greshake et al.) characterises the attacks and establishes how serious they are, but proposes no defence. The second (Liu et al.) supplies evaluation infrastructure and shows that existing defences are fragile, without building a stronger one. The third (Hines et al., Willison) proposes mitigations that either rely on the model choosing to comply, and so fail silently when it does not, or cost so much architecturally that most deployments cannot adopt them.

Three specific gaps follow. First, no defence we surveyed works outside the model while staying model-agnostic, so that swapping the model out does not invalidate it. Second, defences are tested against fixed payload sets, not against an attacker who adapts, so every published robustness figure is an upper bound of unknown tightness. Third, and most concretely, the literature is written and evaluated almost entirely in English. Not one surveyed defence reports cross-lingual coverage, even though most web content is not in English. We measured what that costs: an English-only detector ensemble caught 2.1% of non-English versions of payloads it catches easily in English.

2.3 Problem Statement

An agent reads text from channels of differing trustworthiness and holds tools that can act in the world. Build a control that decides, for each span of text, whether that span is allowed to influence the agent's behaviour. It must do this without a second inference pass in the request path. It must stay valid when the underlying model is swapped out. And when it is uncertain, or when one of its own components breaks, it must fail safe instead of failing silently.

The control also has to be usable. A filter that blocks legitimate work gets uninstalled, so we treat the false-positive rate on benign content as a first-class constraint and build it into the optimisation objective rather than checking it at the end.

2.4 Objectives

Canonicalise hostile text. Reverse zero-width and bidirectional control characters, Unicode compatibility variants, homoglyph substitution, concealed markup and layered encodings, and treat the quantity of reversal required as evidence in its own right, since benign documents require none.

Detect by multiple independent means. Construct an ensemble whose members fail in uncorrelated ways, so that no single evasion technique defeats all of them simultaneously.

Extend detection beyond English. Provide cross-lingual coverage such that an adversary cannot bypass the system merely by changing language.

Fuse evidence under provenance. Combine detections probabilistically and weight the result by the trustworthiness of the channel on which the text arrived.

Neutralise without discarding. Preserve the informational content of suspicious text while destroying its capacity to act as an instruction, so that the agent remains useful.

Constrain capability per turn. Revoke precisely those tools that the detected attack family would require, applying least privilege for the duration of a single turn.

Guard the outbound path. Detect credential disclosure, system-prompt leakage and zero-click exfiltration channels in model output and in proposed tool invocations.

Evaluate honestly. Measure against held-out data, against an adaptive adversary and against a real language model, and report what is measured including results that do not favour the system.

2.5 Project Plan

The work was divided into seven phases over one semester: literature survey and threat modelling, corpus construction and pipeline design, detector implementation and calibration, held-out evaluation and ablation, cross-lingual extension, adaptive adversary and hardening, and finally the console, the live-model evaluation and this report. All seven are complete. Fig. 1 shows the schedule as a Gantt chart and the months are indicative. The plan below is the schedule as worked rather than as first drafted: phases 5 and 6 were added partway through in response to findings, and that is recorded here because the order of events is itself part of the result.

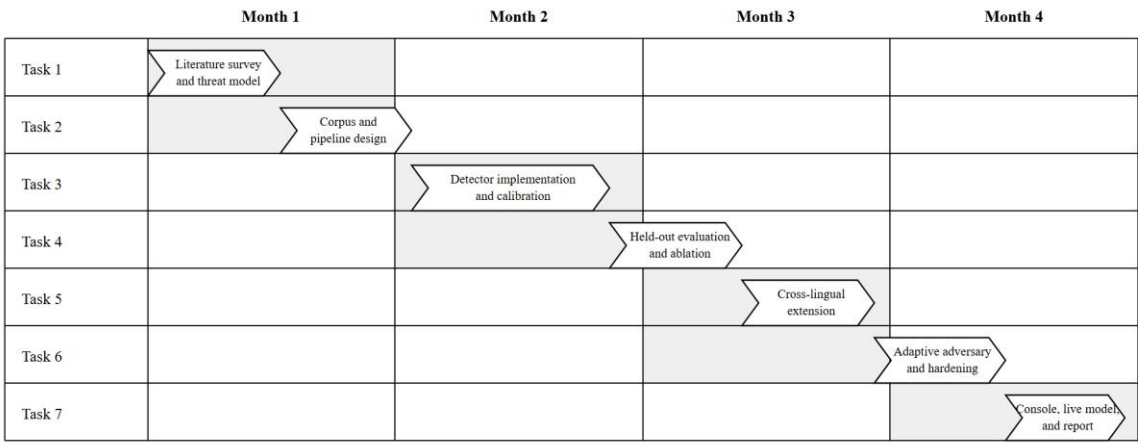


Fig. 1. Gantt chart

Phase	Activity	Weeks	Outcome
1	Literature survey and threat modelling	1–2	Eight attack families identified
2	Corpus construction and pipeline design	3–4	753-sample corpus; five-stage architecture
3	Detector implementation and calibration	5–7	Four detectors; leave-one-family-out calibration
4	Held-out evaluation and ablation	8–9	F1 0.948; benign-class defect identified
5	Cross-lingual extension	10–11	Fifth detector; 12 languages
6	Adaptive adversary and hardening	12–13	Robustness 86.7%; two defects closed
7	Console, live-model evaluation, report	14–15	Web console; end-to-end LLM benchmark

Table 1. Project plan as executed.

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- The system shall accept a span of text together with a declared provenance tier and return a decision of allow, sanitise or block.
- The system shall reverse zero-width characters, Unicode compatibility variants, homoglyph substitutions, concealed markup and base64, hexadecimal, percent and ROT13 encodings prior to inspection.
- The system shall score text over overlapping windows so that a short payload embedded in a long document is not diluted.
- The system shall detect the eight attack families: instruction override, role manipulation, prompt leaking, exfiltration, tool abuse, delimiter escape, social engineering and obfuscation.
- The system shall detect injections expressed in languages other than English.
- The system shall weight its risk assessment by provenance, distinguishing system, user, tool, retrieved and web channels.
- The system shall neutralise suspicious spans by datamarking and nonce fencing while preserving their informational content.
- The system shall revoke, for the duration of one turn, those capabilities associated with the detected attack family.
- The system shall inspect outbound model output for credential material, system-prompt disclosure and zero-click exfiltration channels.
- The system shall validate proposed tool invocations against the capability set remaining for that turn.
- The system shall record every decision, including permissive ones, together with the evidence that produced it.

3.1.2 Non-Functional

- Performance: inspection shall add no more than fifty milliseconds per span at the median on commodity hardware. Measured: 5.644 ms median, 14.953 ms at the ninety-fifth percentile.
- Precision: the false-positive rate on benign traffic shall not exceed five per cent. Measured: 2.9%.
- Portability: the system shall carry no third-party runtime dependency and shall require neither network access nor a graphics processor.
- Model independence: the system shall remain valid when the underlying language model is replaced, requiring no fine-tuning and no provider-specific interface.

- Fail-safe behaviour: a malfunctioning detector shall be treated as a detection rather than ignored, since middleware that silently degrades to inaction is worse than none, the operator continuing to believe themselves protected.
- Auditability: every decision shall carry human-readable justification sufficient to reconstruct it.
- Maintainability: adding an attack signature, an exemplar or a language shall require a data change rather than a code change.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Technology Availability. The system is written entirely against the Python standard library. Every mechanism it needs, Unicode normalisation, regular expressions, hashing and JSON, is already in the runtime.

Technical Expertise. The project needs knowledge of applied NLP and of the attack literature. The classifier is a logistic regression over hashed n-grams, implemented from first principles in about two hundred lines, so no machine-learning framework has to be learned or maintained.

Infrastructure. No graphics processor, no network access and no second inference pass. The trained model occupies 182 kilobytes and loads in milliseconds, so deployment adds nothing to the infrastructure an agent already needs.

Integration. The layers are loosely coupled. The middleware is imported as a package and called at two points, before text reaches the model and before output leaves, so it can be fitted to an existing agent without changing the model or the provider.

3.2.2 Economic Feasibility

Software Cost. None. There is no third-party runtime dependency and therefore no licence fee.

Hardware Cost. None beyond the development laptop. The middleware needs no accelerator and no additional server.

Cost per Request. The middleware adds 5.644 milliseconds of processor time per span and no monetary cost. An adjudicating-model defence would roughly double the inference bill of every request instead.

Return on Investment. One successful exfiltration of credentials from a production agent is expensive in itself and in disclosure, so the trade is favourable under any realistic estimate of how often an incident occurs.

Scaling Cost. A larger deployment needs only processor time proportional to the text inspected. Nothing in the design requires shared state between requests.

3.2.3 Social Feasibility

User Acceptance. A control that blocks legitimate work gets uninstalled in its first week. The false-positive rate is therefore part of the optimisation objective rather than a figure checked afterwards, and it is reported beside every detection result.

Transparency. Every decision carries the evidence that produced it, including the permissive ones, so a disputed outcome can be examined and explained instead of merely overridden.

Privacy. The system inspects text an agent is about to process on its operator's behalf and performs no monitoring of users. The audit trail records decisions and their evidence and is intended for the operator of the agent.

Ethical Considerations. Detection is reported with its uncertainty and never presented as proof of intent. Results that do not favour the system, including a null result in the end-to-end evaluation, are reported as measured.

Error Impact. A false positive hides legitimate content and a false negative exposes the agent, so the output is decision support within a defence-in-depth posture and not a sole control.

3.3 System Specification

3.3.1 Hardware Specification

Component	Specification
Processor	x86-64, dual core minimum (development: Intel, 8 logical cores)
Memory	4 GB minimum; approximately 60 MB resident for the library
Storage	50 MB for the library, trained model, corpus and benchmark artefacts
Graphics processor	Not required
Network	Not required by the middleware; optional for the live-model evaluation
Monitor	Any display; the console is a browser page and the demonstrations are text

Table 2. Hardware specification.

3.3.2 Software Specification

Component	Specification
Operating system	Windows 11; platform independent (Linux and macOS supported)
Programming language	Python 3.9 or later (developed on 3.11.4)
Runtime dependencies	None — Python standard library only
Development environment	Visual Studio Code; Git
Evaluation tooling	Bundled scripts; Node.js with docx and pptxgenjs for document generation
Optional components	Ollama with a local instruct model, for end-to-end evaluation

Component	Specification
Libraries and frameworks	None at runtime. Node.js with docx and pptxgenjs at build time only, for generating this report and the slides
Database	None. Trained weights and calibration are JSON; the corpora are JSONL; the audit trail is held in memory for the session
Security tools	Input size limits, loopback-only binding, no-store response headers, X-Content-Type-Options, and the project's own egress guard and capability gate
Interface	Python API; command-line demonstrations; local web console

Table 3. Software specification.

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

Fig. 2 shows the layered architecture. The presentation layer collects a span of text together with the channel it arrived on and sends a JSON request to the service layer, which validates it and calls the middleware core. The core runs the span through five stages in order: canonicalisation, segmentation into overlapping windows, scoring by five independent detectors, evidence fusion with a policy decision, and neutralisation. Only then does any part of the span reach the language model. On the return path the egress guard inspects the model's reply and validates any tool call it proposes against the capabilities that survived the decision. The detection layer draws its thresholds and weights from read-only JSON artefacts in the model and data layer. The offline training path, drawn dashed because it never runs in the request path, produces those artefacts. Table 4 maps each layer to its modules.

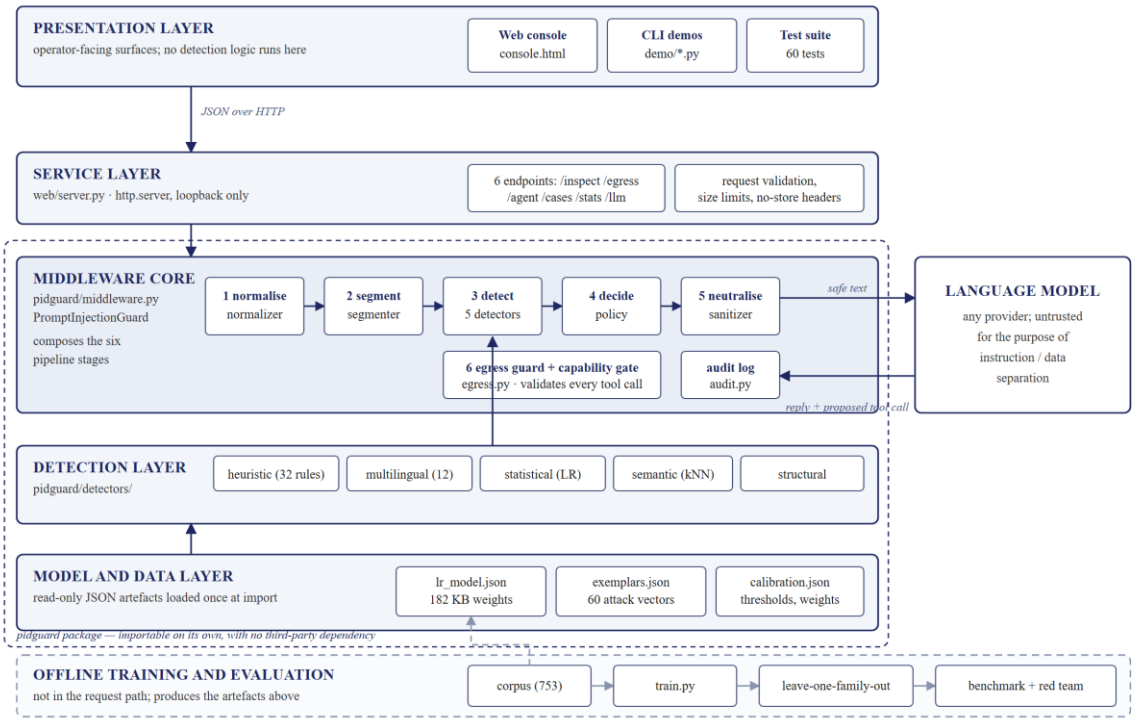


Fig. 2. System Architecture

Layer	Files / modules	Responsibility
Presentation	web/console.html; demo/showcase.py, demo_agent.py, try_it.py; tests/test_pidguard.py	Collects the text and its trust tier, sends the request, and shows the decision with its evidence. No detection logic runs here, so no verdict shown in the browser is computed in the browser.
Service	web/server.py	Validates requests and enforces size limits; exposes six JSON endpoints over the Python standard-library HTTP server; binds to the loopback interface only.

Layer	Files / modules	Responsibility
Middleware core	pidguard/middleware.py, normalizer.py, segmenter.py, policy.py, sanitizer.py, egress.py, audit.py, types.py, config.py	Composes the six pipeline stages behind one entry point, PromptInjectionGuard. Holds the trust tiers, the fusion rule, the decision thresholds, the capability gate and the audit trail.
Detection	pidguard/detectors/base.py, heuristic.py, multilingual.py, statistical.py, semantic.py, structural.py	Five independent mechanisms behind a common interface: 32 signature rules, a 12-language lexicon, a logistic-regression classifier, a nearest-neighbour index over 60 exemplars, and content-blind structural analysis.
Model and data	pidguard/models/lr_model.json, exemplars.json, calibration.json; data/corpus_train.jsonl, corpus_test.jsonl, corpus_multilingual.jsonl	Trained weights, the exemplar index and the calibrated thresholds, loaded once at import. Corpora are held as JSONL so every sample can be inspected directly.
Offline training and evaluation	data/build_corpus.py, build_multilingual.py; benchmark/train.py, evaluate.py, evaluate_multilingual.py, redteam.py, evaluate_llm.py, llm_sanity.py	Builds the corpora, fits the classifier, selects every threshold by leave-one-family-out cross-validation, and runs the held-out, cross-lingual, adversarial and end-to-end evaluations. None of this is in the request path.

Table 4. Layers, files, and responsibilities

The design decision everything else rests on is that trust belongs to the channel the text arrived on, not to the text. Five tiers are distinguished, each carrying a multiplier applied to the fused risk score. The same sentence is therefore treated differently depending on where it came from: a user is entitled to write “ignore my previous message”, and a scraped web page is not. Table 5 gives the tiers.

Trust tier	Multiplier	What arrives on this channel
SYSTEM	0.00	The developer's own prompt. Never inspected, because it is supposed to contain instructions.
USER	0.75	What the operator typed. Trusted, but not exempt: an operator can paste in something they were sent.
TOOL	1.15	Output returned by a tool the agent called.
RETRIEVED	1.25	A document fetched from a store or index.
WEB	1.40	Content fetched from the open internet. Trusted least.

Table 5. Trust tiers and their risk multipliers

4.2 Design

Fig. 3 summarises the decision logic that the two algorithms below implement. A span is inspected unless it came from the developer's own prompt, which is trusted by construction. It is then canonicalised, cut into overlapping windows and scored by five detectors. The scores are fused, weighted by the trust tier and compared against two thresholds, which

yields one of three outcomes. Independently of that outcome, every tool associated with a detected attack family is revoked for the duration of the turn.

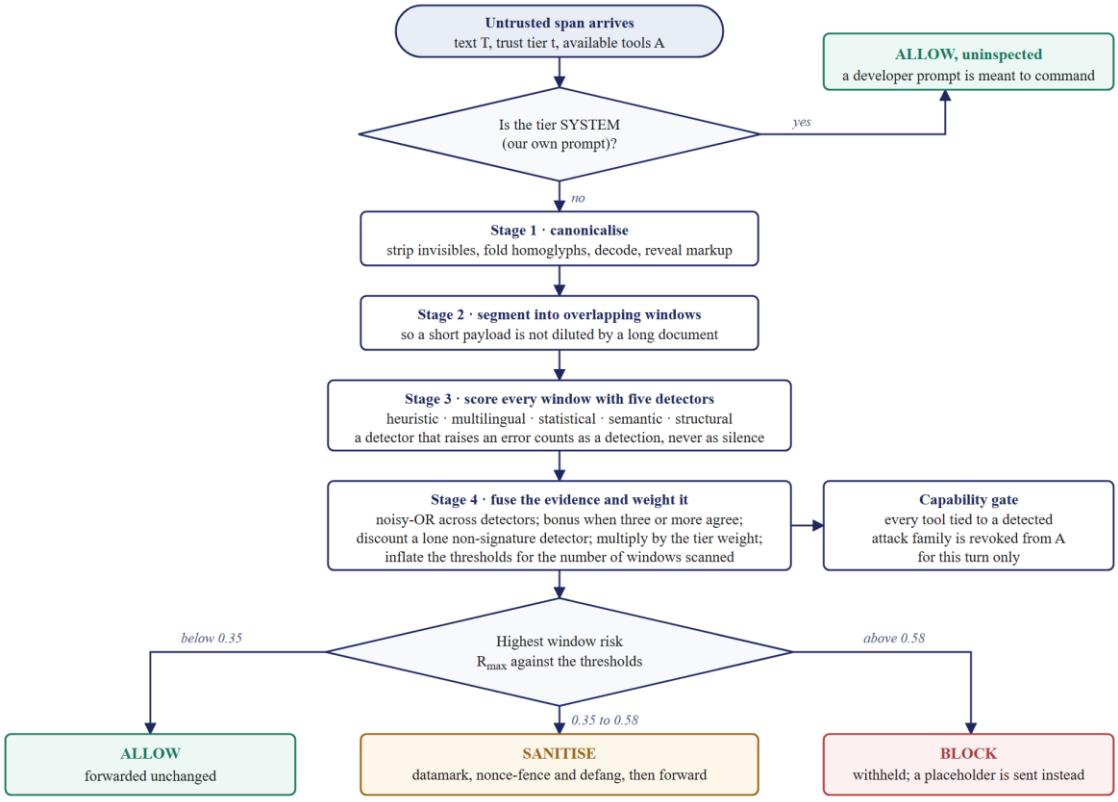


Fig. 3. Decision logic and the three outcomes

4.2.1 Algorithm 1 — Windowed Detection and Risk Fusion

This algorithm decides whether a span of text is allowed to influence the agent. Two parts of it are not obvious. Windowing stops a short payload from being diluted by a long, innocent document. The multiple-comparisons correction then stops the resulting maximum-over-windows statistic from pushing up false positives as documents get longer, which is the price windowing would otherwise charge.

1. Input: raw text T, trust tier t, available tool set A.

2. If t is SYSTEM, return ALLOW; the developer's own prompt is not inspected.

3. R ← Canonicalise(T): strip zero-width and bidirectional controls; apply Unicode NFKC; fold homoglyphs within mixed-script tokens only; reveal hypertext comments and concealed spans, hoisting their content onto its own line; decode base64, hexadecimal, percent and ROT13 payloads. Record every transformation applied.

4. W ← Segment(R): split into units at sentence, line and block boundaries; group into overlapping windows of at most 120 characters with one unit of overlap.

5. For each window w in W: for each detector d in {heuristic, multilingual, statistical, semantic, structural}, collect detections, each carrying a severity and a confidence. A detector that raises an exception contributes a medium-severity detection rather than silence.

6. For each window, fuse within each detector by noisy-OR over severity × confidence, cap the per-detector contribution, weight by detector, then fuse across detectors by noisy-OR.

7. Apply an agreement bonus when three or more detectors concur, and a discount when a lone non-signature detector fires without corroboration.

8. Multiply by the tier multiplier $\tau(t)$ to obtain the window risk.

9. Let Rmax be the greatest window risk and c the concentration, (Rmax − mean of the remainder) / Rmax.

10. Inflate both decision thresholds by $(1 + k \cdot \log_2|W| \cdot (1 - c))$, a discrete Bonferroni analogue damped by concentration so that padding a document cannot purchase leniency.

11. Return BLOCK if Rmax exceeds the inflated block threshold, SANITISE if it exceeds the inflated sanitise threshold, otherwise ALLOW; and revoke from A every capability associated with a detected attack family whose evidence exceeds the gating floor.

We fuse with noisy-OR instead of a vote for two reasons. Several weak but independent signals ought to be able to add up to a decision, and one detector firing on the same match repeatedly ought not to be able to saturate the score. The concentration term carries information of its own. An innocent document is evenly unremarkable all the way through. A poisoned one has a single region that scores far above everything around it.

4.2.2 Algorithm 2 — Neutralisation and Egress Control

If we blocked every suspicious document we would end up with an agent that refuses to read the web, which nobody would use. The second algorithm keeps the information and destroys only its ability to give orders. It then guards the outbound path on the working assumption that the first algorithm may have missed something.

1. Input: canonical text R, tier t, detections D, the model's eventual response S and any proposed tool call (n, args).

2. Datamark: interleave a private invisible separator (U+2063) into every whitespace position of R, so that the span no longer tokenises as a conversational turn while remaining readable.

3. Fence: wrap the marked span in delimiters carrying a fresh 64-bit random nonce, so that an adversary cannot close the fence without guessing it.

4. Defang: rewrite each detected span as inert quoted evidence, so that both the agent and a later auditor can see what was attempted.

5. Prepend a provenance banner stating the source tier and the detected attack families; the integrator adds the corresponding security contract to the system prompt once.

6. On egress, scan S for credential material against eight patterns, for verbatim reproduction of protected system-prompt text, and and for zero-click beacons, meaning markdown images whose host is not allow-listed or whose query carries a data-bearing value. Redact each match.

7. Validate (n, args) against the capability set surviving step 11 of Algorithm 1; reject the call if n was revoked, and recursively scan args as untrusted output in their own right.

4.2.3 The Detector Ensemble

We chose the five detectors so that they fail in uncorrelated ways. Each one asks a different question about the same piece of text, which means a single evasion trick does not defeat all five at once.

Detector	Question answered	Mechanism
Heuristic	Does this match a known attack signature?	32 regular-expression rules across 8 families, matched over up to three text views, with a fuzzy near-miss fallback
Multilingual	Is this an injection in another language?	Lexicon over 12 languages; fires only when two compatible categories co-occur within one language
Statistical	How injection-like is this statistically?	Hashed word 1–2-grams and character 4-grams into 2^{15} buckets; L2-regularised logistic regression trained by SGD
Semantic	Which known attack is this closest to?	Cosine similarity to an index of 60 exemplars, yielding an auditable nearest neighbour
Structural	Does this behave like a command aimed at an agent?	Content-blind: clause-initial imperative density, second-person adjacent to obligation, fenced blocks, outbound URL shape, entropy

Table 6. The five detectors and their complementary failure modes.

5. EXPERIMENTAL SETUP AND OUTPUT

5.1 Experimental Setup

We ran every experiment on an ordinary laptop (x86-64, 8 logical cores, 16 GB memory, Windows 11) under Python 3.11.4, with no third-party runtime dependency. All timings are single-threaded and cover the complete pipeline, not one stage of it.

The corpus is 753 samples we wrote ourselves, split into 528 for training and 225 held out for testing. We applied four safeguards, each aimed at a specific way an evaluation like this can flatter itself:

- Test seeds are phrased independently of training seeds, so that no template is shared between splits and the reported figures measure generalisation rather than memorisation.
- Splitting is grouped by seed identifier, so that every obfuscated variant of a sentence remains in the same fold; without this an augmented near-duplicate leaks across the boundary and every figure is optimistic.
- The test file is never opened by any training or calibration script.
- Pipeline-level parameters were chosen by leave-one-family-out cross-validation. A whole attack category is withheld from the classifier and the exemplar index, and recall is then measured on that unseen category. We did this because a conventional validation split saturated near 1.0 and could not tell two settings apart.

We ran three further evaluations beyond the held-out set. A cross-lingual benchmark of 108 samples spans 12 languages, comprising 48 attacks and 60 hard negatives, which are foreign-language documents using the same vocabulary as the attacks. An adaptive adversary runs a hill-climbing evasion search over fourteen mutation operators with a budget of forty queries per payload. An end-to-end evaluation puts a local instruct model through a real agent loop under three conditions.

Throughout, the objective we optimised was F1 minus 0.55 times the false-positive rate, subject to a floor on recall. False positives are inside the objective, not used as a tie-breaker afterwards, because a filter that blocks legitimate work gets uninstalled.

5.2 Output of the Proposed System

Table 7 gives the ablation on the held-out English set. Each configuration is a strict superset of the one above it, so every row shows what that one extra layer bought.

Configuration	Precision	Recall	F1	FPR
Keyword blocklist (baseline)	0.400	0.023	0.043	0.022
Heuristic only	1.000	0.598	0.748	0.000
Statistical only	1.000	0.310	0.474	0.000
Semantic only	0.000	0.000	0.000	0.000

Configuration	Precision	Recall	F1	FPR
Structural only	1.000	0.034	0.067	0.000
Heuristic + statistical	1.000	0.736	0.848	0.000
Heuristic + stat + semantic	0.953	0.931	0.942	0.029
Full ensemble	0.953	0.943	0.948	0.029

Table 7. Ablation on the held-out set (225 samples: 87 attack, 138 benign).

Two things in this table are worth pointing out precisely because they do not flatter us. The semantic and structural detectors are weak on their own, reaching F1 of 0.000 and 0.067 respectively. They earn their place by corroborating the others. Adding the semantic detector to the heuristic and statistical pair lifts recall from 0.736 to 0.931. They also attach readable evidence to a decision, which a bare probability cannot. The keyword baseline reaches only 2.3% recall. That is not a rigged comparison, it is the point being demonstrated: the test attacks were deliberately written away from stock wording, which is exactly what a real attacker would do.

Attack family	Recall
delimiter escape	1.000
exfiltration	1.000
obfuscation	1.000
prompt leak	1.000
social engineering	1.000
tool abuse	1.000
instruction override	0.867
role manipulation	0.750

Table 8. Recall by attack family. Role manipulation remains the weakest category.

Evasion technique	Recall
b64 wrap	1.000
homoglyph	1.000
spaced	1.000
zero width	1.000
plain	0.931
wrap indirect	0.909
leetify	0.667

Table 9. Recall by evasion technique. The wrap-indirect row is the indirect-injection case: a payload embedded in an otherwise benign document.

Measure	Value
Median added latency	5.644 ms

Measure	Value
Mean added latency	6.727 ms
95th percentile	14.953 ms
99th percentile	17.513 ms
Runtime dependencies	0

Table 10. Latency, single-threaded, complete pipeline.

6. RESULT ANALYSIS

6.1 Comparison with Existing Systems

Table 11 compares our system with the defences surveyed in Section 2. Where a direct comparison is possible we evaluated on the same held-out set. Where the published work does not allow one, we describe the difference instead of inventing a number.

System	Recall	FPR	Model-agnostic	Fails safe
Keyword blocklist	0.023	0.022	Yes	Yes
Prompt-level instruction (Hines et al.)	n/a — model-dependent	n/a	No	No — silent
Dual LLM pattern (Willison)	n/a — architectural	n/a	Yes	Yes
Signature rules only (ours, ablated)	0.598	0.000	Yes	Yes
Proposed system (PID-Guard)	0.943	0.029	Yes	Yes

Table 11. Comparison with existing approaches.

Two honest qualifications go with that table. A prompt-level defence cannot be given a recall figure that is independent of the model it runs on, and that is exactly the criticism of it: how well it works is a property of the model, not of the defence. The dual LLM pattern is an architecture, not a classifier, so there is no comparable measurement to make. It does carry a cost we do not, namely a second inference pass on every request.

6.1.1 Cross-Lingual Evaluation

The literature we surveyed is written and evaluated in English. When we ran the same payloads through our ensemble in other languages, we found a complete bypass. An attacker did not need any clever technique. They only had to change language.

Configuration	Recall	FPR on controls
English-only ensemble	2.1%	0.000
With cross-lingual detector	100.0%	0.000

Table 12. Cross-lingual benchmark: 108 samples, 12 languages, 60 hard negatives using the same vocabulary as the attacks.

The hard negatives are the part of this result that matters. Foreign-language documents containing the words for password, settings, system and instructions are not blocked. A detection needs two compatible categories to occur together inside one language, and it needs at least one matched word to be distinctive to that language instead of a cognate it shares with English. Without that second condition the detector fires on any French page that mentions instructions.

6.1.2 Adaptive Adversary

A static benchmark only measures the attacks its authors thought of. To get a bound against an attacker who adapts, we wrote a hill-climbing search that mutates each payload we currently catch, keeps whichever mutation lowered the risk score, and repeats until it either evades or uses up a budget of forty queries.

Measure	Value
Robustness (40-query budget)	86.7%
Evasion rate	13.3%
Mean queries per successful evasion	15.38
Payloads never defeated	52 of 60

Table 13. Adaptive adversary. The search is white-box, meaning it reads our exact risk score, so the evasion rate is an upper bound on what a black-box attacker achieves and the query count a lower bound on their effort.

This evaluation improved the system more than any other. It found two defects that every static test had missed. The first was in our ROT13 handling: we only accepted a decode when the plaintext happened to contain one of five hardcoded keywords, and that alone accounted for most successful evasions. The second was that hiding a payload inside an HTML comment succeeded in a single mutation, because the revealed instruction was averaged in with the surrounding markup. We replaced the keyword check with a language-likeness test, on the reasoning that an attacker picks the words but cannot stop plaintext reading like language while ciphertext does not. For the second we hoist concealed content into a scan window of its own. Robustness went from 21.7% to 86.7%.

6.1.3 End-to-End Evaluation Against a Language Model

Everything above measures the filter. The claim that actually matters is whether an agent behind this middleware gets hijacked, and answering that needs a real model. We ran a local instruct model (qwen2.5:1.5b-instruct) was run through a genuine agent loop over 12 poisoned and 6 clean documents at temperature zero, under three conditions.

Condition	Attack success
A — naive agent, no defensive instruction	0%
B — prompt-level defence in the system prompt	0%
C — PID-Guard middleware	0%
Poisoned documents withheld before the model	83%
Utility on clean documents (A → C)	100% → 100%

Table 14. End-to-end evaluation. Attack success is behavioural: the model invoked a dangerous tool.

This model resisted every payload even with no defensive instruction at all, so there is no difference between conditions to claim and we do not claim one. What the figures do support is narrower and still worth having. The guard withheld 83% of poisoned documents before

the model ever saw them, and clean-document utility was unchanged, which is the usual place a filter goes wrong. We report the result as we measured it.

A zero is also exactly what a broken harness reports, so we built a positive control in which the operator legitimately asks for each dangerous action. The model calls the file-reading and mail-sending tools and our parser recognises both, which establishes that the measurement could have seen a hijack if one had happened. Writing that control exposed two defects in our own measurement code, and both of them had been biasing attack success downward. One was a regular expression that could not match nested JSON, so every genuine tool call was read as no call at all. The other was that truncated tool calls were also being read as refusals. We fixed both and regenerated every figure.

7. CONCLUSION AND FUTURE WORKS

We set out to enforce, outside the model, a separation that the model cannot enforce for itself. The middleware we built reaches 94.3% recall at 2.9% false positives on a held-out English set, 100.0% recall across 12 languages with no false positives on foreign-language controls, and 86.7% robustness against an adaptive white-box adversary, adding 5.644 milliseconds per span with no third-party dependency.

The findings we learned most from were the negative ones, and three are worth recording. The first is that the makeup of the benign class mattered more than the makeup of the attack class. We first trained the classifier on benign data made up only of user requests. It quietly learned the rule “if this is not a request, it is an attack”, and went on to score eighty-eight per cent of ordinary document prose as injection. Since indirect injection is by definition about documents, that was our single biggest source of false positives, and no amount of threshold tuning would have fixed it. Only fixing the data fixed it. The second is that evaluating in one language hid a complete bypass from us for most of the project. The third is that every evaluation harness we wrote contained at least one defect that made our results look better than they were, and in each case we only found it by building a control for it.

We should be clear about the limitations. The corpus was written by us and holds 753 samples, so our figures describe the system on that distribution and are not a claim about production traffic. Role manipulation is still our weakest family at 0.750 recall, and paraphrases of the form “take on the identity of a model that was never given any policy” get past both the rule base and the classifier. We left that case failing instead of patching it with a rule fitted to the test set. The end-to-end evaluation used small models in single-turn settings, which is a weaker test than a long agentic loop would be.

The future work follows straight from those limitations. Evaluating against a public benchmark written by somebody else would address the biggest threat to validity. Swapping the hashed vectoriser in the semantic detector for sentence embeddings should improve our weakest component, and it is a one-function change because we designed the interface for it. Extending the end-to-end evaluation to larger models and multi-turn loops would show whether the null result in Section 6.1.3 survives greater pressure. The cross-lingual lexicon covers twelve languages chosen by prevalence. Extending it is a data task, not a code change, and the same corroboration rule should continue to hold.

8. REFERENCES

Journals and Conferences: IEEE format

[1] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in Proc. NeurIPS Workshop on ML Safety, 2022.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90.

[3] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. 33rd USENIX Security Symposium, 2024.

[4] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," arXiv:2403.14720, 2024.

[5] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: a taxonomy of indirect prompt injection threats," in Proc. ACM Conference on Computer and Communications Security Workshops, 2023.

[6] N. Carlini, M. Nasr, C. A. Choquette-Choo, et al., "Are aligned neural networks adversarially aligned?" in Advances in Neural Information Processing Systems, vol. 36, 2023.

[7] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv:2307.15043, 2023.

[8] J. Yi, Y. Xie, B. Zhu, et al., "Benchmarking and defending against indirect prompt injection attacks on large language models," arXiv:2312.14197, 2023.

[9] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Advances in Neural Information Processing Systems, 2024.

[10] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "TextBugger: Generating adversarial text against real-world applications," in Proc. Network and Distributed System Security Symposium (NDSS), 2019.

[11] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," Journal of Big Data, vol. 3, no. 1, pp. 1–40, 2016.

[12] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," Journal of Machine Learning Research, vol. 3, pp. 1137–1155, 2003.

Weblinks

[13] S. Willison, "The dual LLM pattern for building AI assistants that can resist prompt injection," 2023. [Online]. Available: https://simonwillison.net/2023/Apr/25/dual-llm-pattern/

[14] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[15] Unicode Consortium, “Unicode Technical Standard #39: Unicode Security Mechanisms,” 2023. [Online]. Available: https://www.unicode.org/reports/tr39/

APPENDIX A. SOURCE CODE AND REPOSITORY

A.1 Repository

The complete working code accompanies this report. It is self-contained: the middleware has no third-party runtime dependency, so the repository can be cloned and run with nothing installed beyond Python 3.9 or later.

Item	Value
Code submitted	The complete source tree accompanies this report. The core listings are reproduced in Appendix B
Repository	Not published; source submitted with this report
Language	Python 3.9 or later, developed on 3.11.4
Runtime dependencies	None. Python standard library only
Total size	about 3,170 lines in the middleware package, 680 lines of tests
Licence	Submitted for academic evaluation

Table 15. Repository details.

A copy of the full source tree is also submitted alongside this report, so the code can be inspected without network access.

A.2 Directory Layout

Path	Contents
pidguard/	The middleware package. One module per pipeline stage, with the five detectors in a subpackage behind a common interface
pidguard/detectors/	heuristic.py, multilingual.py, statistical.py, semantic.py, structural.py and the shared base interface
pidguard/models/	Trained artefacts: lr_model.json (classifier weights), exemplars.json (60 attack vectors), calibration.json (thresholds and detector weights)
data/	Corpus builders and the generated corpora as JSONL: 528 training, 225 held-out test, 108 cross-lingual
benchmark/	Training, held-out evaluation, cross-lingual evaluation, the adaptive adversary, the end-to-end LLM evaluation and its positive control
tests/	test_pidguard.py: 60 tests covering every stage, including regression tests for defects found during development
web/	server.py and console.html: the local demonstration console, which runs the real pipeline
demo/	Command-line demonstrations and the 50 verified attack cases used by the console
report/, ppt/, paper/	Generators for this report, the slides and the conference paper. Every figure is read from the benchmark output at build time

Table 16. Directory layout of the submitted code.

A.3 Running the Code

Three commands reproduce the central claims of this report. No installation step is required and no network access is used.

```
python tests/test_pidguard.py
```

Runs the 60-test suite over every stage of the pipeline. Expected output: 60/60 passed.

```
python benchmark/evaluate.py
```

Evaluates the full system on the 225 held-out samples and prints the ablation of Table 7, the recall breakdowns of Tables 8 and 9, and the latency figures of Table 10. Expected output: recall 94.3%, false positives 2.9%, F1 0.948.

```
python demo/showcase.py
```

Runs 20 fixed cases through the pipeline and prints the verdict and evidence for each. Expected output: 20/20 correct.

Two further commands reproduce the remaining evaluations. The cross-lingual benchmark of Section 6.1.1 is reproduced by benchmark/evaluate_multilingual.py, and the adaptive adversary of Section 6.1.2 by benchmark/redteam.py. The end-to-end evaluation of Section 6.1.3 additionally requires a local Ollama instance and is reproduced by benchmark/evaluate_llm.py; every other result is reproducible without it.

The demonstration console is started with web/server.py, which serves a local page on the loopback interface. Every verdict shown on that page is computed by the same pipeline the benchmarks use.

APPENDIX B. CORE IMPLEMENTATION

Three listings follow, taken verbatim from the submitted source. They cover the public entry point, the fusion and decision rule, and the classifier, which together carry the contribution described in Section 4. Line numbers are those of the files themselves.

B.1 Pipeline Entry Point

PromptInjectionGuard.inspect is the single public entry point. It runs the five ingress stages in order and returns a result carrying the decision, the evidence, the surviving capability set and the text to hand to the model.

pidguard/middleware.py, lines 81-176

```
 81      def inspect(self, text: str, tier: TrustTier = TrustTier.WEB,
 82                  available_tools: Sequence[str] | None = None) -> GuardResult:
 83          """Inspect one span of untrusted text. This is the hot path."""
 84          started = time.perf_counter()
 85
 86          if tier is TrustTier.SYSTEM and self.policy.config.trust_system_tier:
 87              result = GuardResult(
 88                  decision=Decision.ALLOW, risk_score=0.0, original_text=text,
 89                  safe_text=text, trust_tier=tier,
 90                  allowed_tools=list(available_tools or []),
 91                  latency_ms=(time.perf_counter() - started) * 1000)
 92              self.audit.record(result)
 93              return result
 94
 95          report = normalize(text)
 96          canonical = report.normalized
 97
 98          # Windowed scan: score each region and keep the worst. A payload buried
 99          # in a long benign page stays visible instead of being averaged away.
100          wins = windows(canonical, self.window_chars)
101          window_risks: List[float] = []
102          all_detections: List[Detection] = []
103          worst_detections: List[Detection] = []
104          worst_risk = -1.0
105
106          for win in wins:
107              dets = self._scan_window(win.text, report)
108              for d in dets:                          # make spans document-absolute
109                  if d.span:
110                      d.span = (d.span[0] + win.start, d.span[1] + win.start)
111                  d.metadata["window"] = win.index
112              r = self.policy.fuse(dets, tier)
113              window_risks.append(r)
114              all_detections.extend(dets)
115              if r > worst_risk:
116                  worst_risk, worst_detections = r, dets
117
118          # Localised risk is itself evidence: benign documents are uniformly
119          # benign, injected ones have exactly one hot region.
120          conc = concentration(window_risks)
121          multiplier = 1.0 + self.concentration_bonus * conc
122
123          decision, risk = self.policy.decide(worst_detections, tier, multiplier,
124                                              n_windows=len(wins),
125                                              concentration=conc)
126          # Tool gating uses evidence from anywhere in the document, not just
127          # the hottest window -- an exfil URL and its command may be paragraphs
128          # apart, and either one alone justifies revoking egress.
129          allowed, revoked = self.policy.gate_tools(all_detections, available_tools)
130
131          if decision is Decision.BLOCK:
132              families = ", ".join(sorted({d.metadata.get("family", "unknown")
133                                           for d in all_detections})) or "unclassified"
134              safe_text = BLOCK_PLACEHOLDER.format(tier=tier.value, families=families)
135          elif decision is Decision.SANITIZE:
136              # Sanitise the *canonical* text, not the original: the model should
137              # never receive the obfuscation layer we just stripped, and the
138              # detection spans are only meaningful against canonical text.
139              safe_text = self.sanitizer.sanitize(canonical, tier, worst_detections)
140          else:
141              # Even ALLOW output is fenced when it comes from an untrusted tier:
142              # the cheapest defence is applied unconditionally.
143              safe_text = (self.sanitizer.sanitize(canonical, tier, [])
144                           if tier in (TrustTier.WEB, TrustTier.RETRIEVED, TrustTier.TOOL)
145                           else text)
146
147          result = GuardResult(
148              decision=decision, risk_score=risk, original_text=text,
```

```
            safe_text=safe_text, trust_tier=tier, detections=worst_detections,
            normalization=report, allowed_tools=allowed, revoked_tools=revoked,
            latency_ms=(time.perf_counter() - started) * 1000)
        result.metadata = {"windows": len(wins), "concentration": round(conc, 3),
                           "window_risks": [round(r, 3) for r in window_risks]}
        self.audit.record(result)
        return result

    def _scan_window(self, text: str, report) -> List[Detection]:
        """Run every detector over one window, failing closed on crashes."""
        detections: List[Detection] = []
        for det in self.detectors:
            try:
                detections.extend(det.scan(text, report))
            except Exception as exc:                    # noqa: BLE001
                # A broken detector must not silently create a blind spot:
                # middleware that degrades to a no-op is worse than none,
                # because the operator still believes they are protected.
                if not self.fail_open:
                    detections.append(Detection(
                        detector=det.name, rule_id="ERR-01",
                        description=(f"Detector failed ({type(exc).__name__}); "
                                     "treating input as suspicious"),
                        severity=Severity.MEDIUM,
                        confidence=0.5, metadata={"family": "detector_error"}))
        return detections

    def inspect_many(self, spans: Sequence[tuple],
```

B.2 Evidence Fusion and Decision

PolicyEngine.fuse combines the detector opinions by noisy-OR, applies the agreement bonus and the lone-detector discount, and multiplies by the trust-tier weight. PolicyEngine.decide applies the multiple-comparisons correction and returns one of the three outcomes together with the capability gate.

pidguard/policy.py, lines 101-175

```
    def fuse(self, detections: Sequence[Detection], tier: TrustTier) -> float:
        if not detections:
            return 0.0
        cfg = self.config
        by_detector: Dict[str, List[float]] = {}
        for d in detections:
            by_detector.setdefault(d.detector, []).append(d.weighted_score)

        per_detector: List[float] = []
        for name, scores in by_detector.items():
            weight = cfg.detector_weights.get(name, 1.0)
            # within a detector: noisy-OR, then cap, then weight
            combined = min(cfg.per_detector_cap, _noisy_or(scores))
            per_detector.append(min(1.0, combined * weight))

        fused = _noisy_or(per_detector)

        # Agreement bonus: independent detector families concurring is the
        # strongest evidence we have, and it is what separates a real attack
        # from one noisy regex.
        if len(by_detector) >= 3:
            fused = min(1.0, fused * 1.12)
        elif len(by_detector) == 1:
            solo = next(iter(by_detector))
            fused *= (0.88 if solo in ("heuristic", "multilingual")
                      else cfg.solo_penalty)

        return min(1.0, fused * tier.weight)

    # ------------------------------------------------------- decision ----
    def decide(self, detections: Sequence[Detection], tier: TrustTier,
               risk_multiplier: float = 1.0, n_windows: int = 1,
               concentration: float = 0.0) -> tuple:
        """Return (Decision, risk_score).

        `risk_multiplier` folds in document-level evidence no single detector
        can see (the risk-concentration bonus). `n_windows` and
        `concentration` drive the
        multiple-comparisons correction described on PolicyConfig.
        """
        cfg = self.config
        if tier is TrustTier.SYSTEM and cfg.trust_system_tier:
            return Decision.ALLOW, 0.0

        risk = min(1.0, self.fuse(detections, tier) * risk_multiplier)

        # Hard block: an unambiguous critical signature from an untrusted
        # channel does not need corroboration. Signature rules measured 0.0
        # FPR, so they are exempt from the multiple-comparisons correction.
```

```
        if tier is not TrustTier.USER:
            for d in detections:
                if (d.severity is Severity.CRITICAL
                        and d.confidence >= cfg.hard_block_confidence
                        and d.detector in ("heuristic", "multilingual")):
                    return Decision.BLOCK, max(risk, 0.9)

        # Multiple-comparisons correction, damped by how concentrated the risk
        # is.
        #
        # The plain correction was an attack surface. Our adaptive-adversary
        # search found that padding a payload with benign prose was the single
        # most potent evasion operator: more filler means more windows, more
        # windows means a higher threshold, so the attacker buys leniency by
        # the paragraph.
        #
        # The resolution is that the two cases look different. A long *benign*
        # document is uniformly mildly suspicious -- low concentration -- and
        # genuinely needs the correction, because it really is N independent
        # chances to trip. A padded *attack* has one hot window surrounded by
        # nothing -- high concentration -- and needs no correction at all; the
        # dilution is itself the evidence. So the correction is scaled by
        # (1 - concentration), which keeps the false-positive benefit on prose
        # while removing the discount the attacker was buying.
        conc = max(0.0, min(1.0, concentration))
        inflation = 1.0 + cfg.window_penalty * math.log2(max(1, n_windows)) * (1.0 - conc)
```

B.3 Feature Hashing and the Classifier

The learned detector is implemented from first principles rather than taken from a package, which is what allows the project to carry no runtime dependency. featurize hashes word 1-2-grams and character 4-grams into 2^15 buckets with sublinear term frequency and L2 normalisation; predict_proba is the logistic model over those features.

pidguard/detectors/statistical.py, lines 44-110

```
def featurize(text: str) -> Dict[int, float]:
    """Hashed bag of word 1/2-grams + char 4-grams, L2-normalised."""
    counts: Dict[int, float] = {}

    def bump(key: str, weight: float = 1.0) -> None:
        idx = hash_str(key) % N_BUCKETS
        counts[idx] = counts.get(idx, 0.0) + weight

    toks = tokenize(text)
    for t in toks:
        bump("w:" + t)
    for a, b in zip(toks, toks[1:]):
        bump("b:" + a + " " + b, 1.2)     # bigrams carry the imperative shape
    low = re.sub(r"\s+", " ", text.lower())
    for i in range(len(low) - 3):
        bump("c:" + low[i:i + 4], 0.35)   # survives character-level evasion

    # sublinear scaling then L2 normalisation -> length-invariant vectors
    for k in counts:
        counts[k] = 1.0 + math.log(counts[k])
    norm = math.sqrt(sum(v * v for v in counts.values())) or 1.0
    return {k: v / norm for k, v in counts.items()}


def hash_str(s: str) -> int:
    """FNV-1a: stable across processes, unlike Python's salted hash()."""
    h = 0x811C9DC5
    for ch in s.encode("utf-8"):
        h ^= ch
        h = (h * 0x01000193) & 0xFFFFFFFF
    return h


# ------------------------------------------------------------------ model ----
class LogisticRegression:
    """Minimal L2-regularised logistic regression trained with SGD."""

    def __init__(self, n_features: int = N_BUCKETS) -> None:
        self.w: Dict[int, float] = {}
        self.b: float = 0.0
        self.n_features = n_features

    def decision(self, x: Dict[int, float]) -> float:
        return self.b + sum(self.w.get(i, 0.0) * v for i, v in x.items())

    def predict_proba(self, x: Dict[int, float]) -> float:
        z = max(-30.0, min(30.0, self.decision(x)))
        return 1.0 / (1.0 + math.exp(-z))

    def fit(self, rows: Iterable[Tuple[Dict[int, float], int]], *, epochs: int = 40,
            lr: float = 0.5, l2: float = 1e-4, seed: int = 7) -> "LogisticRegression":
```

```
        import random
        rows = list(rows)
        n_pos = sum(y for _, y in rows) or 1
        n_neg = len(rows) - n_pos or 1
        # class weights: never let the majority class dictate the boundary
        w_pos = len(rows) / (2.0 * n_pos)
        w_neg = len(rows) / (2.0 * n_neg)
        rng = random.Random(seed)
        for epoch in range(epochs):
            rng.shuffle(rows)
            step = lr / (1.0 + 0.15 * epoch)          # decaying learning rate
            for x, y in rows:
                p = self.predict_proba(x)
                cw = w_pos if y == 1 else w_neg
                g = cw * (p - y)
                self.b -= step * g
```

The remaining modules follow the same structure and are included in full in the submitted source tree.